

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA DE SISTEMAS

MAESTRÍA EN CIENCIA DE DATOS



INFORME:

STUDENT DROPOUT RISK PREDICTION

INTEGRANTES:

**DARWIN RIGOBERTO MAMANI QUISPE
MELBIS CHINO HUAYCANI
MILTON EDWARD HUMPIRI FLORES**

DOCENTE:

M.Sc. MAYENKA FERNANDEZ CHAMBI

ENTREGA PARCIAL:

**INFORME DETALLADO DEL ENTRENAMIENTO DEL MODELO Y
PUESTA EN PRODUCCIÓN**

Puno, 13 de septiembre de 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN	1
1.1. Planteamiento del problema	1
1.2. Justificación	1
1.3. Objetivos	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
2. DESCRIPCIÓN DEL PROYECTO	2
2.1. Descripción general	2
2.2. Alcance del proyecto	2
2.3. Arquitectura de la solución	2
2.4. Flujo de funcionamiento	3
3. DATASET	3
3.1. Descripción del dataset	3
3.2. Características de los datos	3
3.3. Variables utilizadas	4
3.4. Variable objetivo	4
3.5. Preprocesamiento de datos	4
3.6. División del dataset	4
4. DESARROLLO Y ENTRENAMIENTO DEL MODELO	5
4.1. Selección de algoritmos	5
4.2. Entrenamiento de modelos	5
4.3. Validación del modelo	5
4.4. Optimización de hiperparámetros	6
4.5. Selección del modelo final	6
5. EVALUACIÓN DEL MODELO	7
5.1. Métricas de evaluación	7
5.2. Matriz de confusión	7
5.3. Evaluación de resultados	7
6. DESARROLLO DE LA APLICACIÓN	8

6.1. Tecnologías utilizadas	8
6.2. Arquitectura de software	8
6.3. Integración del modelo de aprendizaje automático	8
6.4. Interfaz de la aplicación	9
6.5. Funcionamiento de la aplicación en inglés	9
7. DESPLIEGUE EN PRODUCCIÓN	9
7.1. Entorno de despliegue	9
7.2. Configuración del entorno	10
7.3. Contenerización	10
7.4. Puesta en producción	10
7.5. Verificación del despliegue	10
8. PLAN DE INTEGRACIÓN CONTINUA Y MANTENIMIENTO DEL MODELO	11
8.1. Integración continua (CI/CD)	11
8.2. Mantenimiento del modelo	11
9. PRUEBAS DE FUNCIONAMIENTO	12
9.1. Diseño de pruebas	12
9.2. Prueba de funcionamiento 1: Health check	12
9.3. Prueba de funcionamiento 2: Predicción sobre casos conocidos	12
9.4. Pruebas adicionales	13
9.5. Resultados de las pruebas	13
10. RESULTADOS	13
10.1. Resultados del modelo	13
10.2. Resultados de la aplicación	14
10.3. Resultados del despliegue	14
10.4. Resultados de la integración continua	14
11. CONCLUSIONES	14
12. RECOMENDACIONES	15
ANEXOS	17

1. INTRODUCCIÓN

1.1 Planteamiento del problema

La deserción estudiantil universitaria es uno de los principales desafíos que enfrentan las instituciones de educación superior. Con frecuencia, los estudiantes en riesgo de abandonar sus estudios no son identificados sino hasta que ya es demasiado tarde para una intervención efectiva por parte de la universidad. Esta situación genera pérdidas económicas para la institución, afecta las tasas de graduación, y representa un impacto negativo en la trayectoria personal y profesional del estudiante.

Ante esta problemática, surge la necesidad de contar con una herramienta que permita anticipar, con base en información disponible desde el momento de la matrícula y durante los primeros semestres, qué estudiantes presentan un mayor riesgo de deserción, de modo que el personal de bienestar estudiantil pueda intervenir de manera oportuna.

1.2 Justificación

El presente proyecto se justifica desde una perspectiva tanto académica como práctica. Académicamente, permite aplicar conceptos del curso de Aprendizaje de Máquina I dentro de un ciclo de vida de software completo: entrenamiento de un modelo, exposición mediante una API, despliegue en un entorno de producción, y validación mediante pruebas automatizadas.

Desde el punto de vista práctico, contar con un sistema de predicción de deserción con explicabilidad (mediante SHAP) permite que las decisiones del modelo no sean una “caja negra”, sino que puedan ser comprendidas y justificadas por el personal encargado de la intervención, lo cual aumenta la confianza y utilidad real de la herramienta.

1.3 Objetivos

1.3.1 Objetivo general

Desarrollar una aplicación de aprendizaje automático que prediga el riesgo de deserción de estudiantes universitarios, integrada en un ciclo de vida de software regular, con despliegue en producción y explicabilidad de sus predicciones.

1.3.2 Objetivos específicos

- Entrenar y optimizar un modelo de clasificación capaz de predecir si un estudiante abandonará sus estudios, a partir de datos demográficos, socioeconómicos y de desempeño

académico.

- Construir una API REST que exponga el modelo entrenado, permitiendo realizar predicciones individuales junto con una explicación de los factores que influyeron en cada resultado.
- Desplegar la aplicación en un entorno de producción accesible públicamente, con despliegue automático desde el repositorio de código.
- Verificar el correcto funcionamiento de la aplicación mediante pruebas automatizadas que cubran los casos de uso principales.

2. DESCRIPCIÓN DEL PROYECTO

2.1 Descripción general

El proyecto consiste en un sistema de predicción de deserción estudiantil construido en tres capas: un modelo Random Forest entrenado con datos históricos de estudiantes, una API REST desarrollada con FastAPI que sirve las predicciones de ese modelo, y una interfaz web en Streamlit pensada para que personal sin formación técnica pueda usar el sistema sin dificultad.

2.2 Alcance del proyecto

Esta entrega corresponde a la Unidad I del curso. El trabajo realizado hasta este punto incluye el entrenamiento y optimización del modelo, la construcción de la API y su integración con dicho modelo, el desarrollo de una interfaz bilingüe en inglés y español, el despliegue de la aplicación en un entorno de producción, y la implementación de pruebas automatizadas.

Los procesos de integración continua y de mantenimiento del modelo en producción, que incluyen reentrenamiento, versionamiento y monitoreo, no forman parte de esta entrega. Se abordan de manera preliminar en las secciones 8 y 9, y se desarrollarán con mayor profundidad para la entrega final de la Unidad II.

2.3 Arquitectura de la solución

La solución se organiza en tres componentes. El primero es el modelo, entrenado sobre el conjunto de datos de estudiantes y guardado como archivo serializado (`modelo_desercion.pkl`), junto con el orden de columnas que espera durante la inferencia (`columnas.pkl`). El segundo componente es la API, construida con FastAPI, que carga el modelo al iniciar y expone el

endpoint `POST /predict`. Este endpoint recibe los datos de un estudiante, genera la predicción y calcula la explicabilidad SHAP correspondiente. El tercer componente es el frontend, una aplicación Streamlit que se comunica con la API mediante peticiones HTTP y presenta los resultados a través de un formulario organizado por secciones, un medidor de riesgo y un gráfico de los factores más influyentes.

2.4 Flujo de funcionamiento

El flujo comienza cuando el usuario completa el formulario en la interfaz de Streamlit, ya sea ingresando los datos manualmente o cargando uno de los ejemplos predefinidos. Al confirmar la predicción, el frontend construye un objeto JSON con los 34 campos requeridos y lo envía mediante una petición POST a la API desplegada en Railway. La API valida los datos recibidos, ejecuta el modelo, calcula los valores SHAP y determina el nivel de riesgo correspondiente a la probabilidad obtenida. El resultado, que incluye la predicción, la probabilidad, el nivel de riesgo y los tres factores más influyentes, se devuelve en formato JSON y es presentado por el frontend mediante un medidor visual y un gráfico de barras, en el idioma que el usuario haya seleccionado.

3. DATASET

3.1 Descripción del dataset

El conjunto de datos utilizado es *Predict Students' Dropout and Academic Success* (Realinho et al., 2021), publicado en el repositorio UCI Machine Learning bajo licencia CC BY 4.0. Fue recolectado por el Instituto Politécnico de Portalegre (Portugal) a partir de varias bases de datos institucionales relacionadas con estudiantes de distintas carreras de pregrado, como agronomía, diseño, enfermería, periodismo, gestión y tecnologías.

El conjunto contiene 4424 registros de estudiantes y 35 columnas (34 características más la variable objetivo), sin valores nulos, lo que simplificó considerablemente el trabajo de limpieza de datos.

3.2 Características de los datos

Las variables del dataset se agrupan en cuatro categorías:

- **Demográficas:** estado civil, género, edad al matricularse, nacionalidad, si el estudiante es desplazado o internacional.

- **Socioeconómicas:** calificación y ocupación de los padres, si el estudiante es deudor, si tiene matrícula al día, si cuenta con beca.
- **Académicas:** modalidad y orden de postulación, calificación previa, carrera, turno, y unidades curriculares matriculadas, evaluadas y aprobadas en el primer y segundo semestre, junto con las notas obtenidas.
- **Macroeconómicas:** tasa de desempleo, tasa de inflación y PBI correspondientes al periodo de estudio.

3.3 Variables utilizadas

Se utilizaron las 34 características originales del dataset, sin eliminar ninguna, ya que el análisis de importancia SHAP permite identificar cuáles resultan más relevantes sin necesidad de descartar información desde el preprocesamiento.

3.4 Variable objetivo

El dataset original define tres clases en su variable `Target`: Graduate (2209 estudiantes), Dropout (1421 estudiantes) y Enrolled (794 estudiantes). Para efectos de esta aplicación, el problema se transformó en una clasificación binaria: Dropout frente a No Dropout (que agrupa a Graduate y Enrolled), ya que el objetivo práctico es identificar estudiantes en riesgo de abandonar, sin distinguir entre quienes ya culminaron o quienes continúan matriculados.

Con esta transformación, aproximadamente el 32 % de los registros corresponde a la clase Dropout, lo que refleja un desbalance moderado de clases.

3.5 Preprocesamiento de datos

Dado que el dataset no presenta valores nulos, el preprocesamiento se concentró en dos aspectos: la conversión de la variable `Target` a clasificación binaria, y el uso del parámetro `class_weight="balanced"` durante el entrenamiento para compensar el desbalance entre clases, en lugar de aplicar técnicas de sobremuestreo o submuestreo.

3.6 División del dataset

El conjunto de datos se dividió en entrenamiento y prueba utilizando muestreo estratificado, de manera que la proporción de estudiantes Dropout y No Dropout se mantuviera equivalente en ambos subconjuntos.

4. DESARROLLO Y ENTRENAMIENTO DEL MODELO

4.1 Selección de algoritmos

Se compararon tres algoritmos de clasificación: Regresión Logística, Random Forest y XGBoost. Antes de la optimización de hiperparámetros, los tres modelos se entrenaron con su configuración base y se evaluaron sobre el conjunto de prueba, obteniendo los siguientes resultados:

Modelo	Accuracy	Precision	Recall	F1	CV F1 (5-fold)
Random Forest	0.8825	0.8234	0.8080	0.8156	0.7848
Regresión Logística	0.8750	0.7794	0.8525	0.8143	0.7879
XGBoost	0.8750	0.7946	0.8244	0.8092	0.7799

Se eligió Random Forest como modelo final. Aunque la Regresión Logística obtuvo un F1 ligeramente superior en validación cruzada (0.7879 frente a 0.7848), la diferencia es mínima, y Random Forest ofrece una ventaja decisiva para este proyecto: es compatible con `shap.TreeExplainer`, que calcula explicaciones exactas y rápidas por cada predicción individual. Esta capacidad de explicabilidad es un requisito central de la aplicación, ya que cada predicción debe mostrar los factores que la influenciaron.

4.2 Entrenamiento de modelos

Los tres modelos se entrenaron sobre el 80 % de los datos (conjunto de entrenamiento), utilizando el 20 % restante como conjunto de prueba, dividido mediante muestreo estratificado para conservar la proporción original de estudiantes Dropout y No Dropout. En todos los casos se aplicó `class_weight="balanced"` para compensar el desbalance de clases.

4.3 Validación del modelo

La validación se realizó mediante validación cruzada de 5 folds ($cv=5$), optimizando la métrica F1-score en lugar de accuracy, debido al desbalance de clases presente en el dataset (aproximadamente 32 % de casos Dropout). Esta métrica evita que el modelo obtenga buenos resultados simplemente prediciendo la clase mayoritaria.

4.4 Optimización de hiperparámetros

Se utilizó `GridSearchCV` para optimizar los hiperparámetros de Random Forest, explorando las siguientes combinaciones:

- `n_estimators`: 200, 300, 400
- `max_depth`: 8, 12, sin límite
- `min_samples_leaf`: 1, 3, 5

La mejor configuración encontrada fue:

- `n_estimators = 200`
- `max_depth = 12`
- `min_samples_leaf = 3`
- Mejor F1-score en validación cruzada: **0.7879**

4.5 Selección del modelo final

El modelo final (Random Forest con los hiperparámetros optimizados) se evaluó sobre el conjunto de prueba, obteniendo los siguientes resultados para la clase Dropout:

- Precision: 0.80
- Recall: 0.82
- F1-score: 0.81

Un recall de 0.82 significa que el modelo identifica correctamente al 82 % de los estudiantes que efectivamente abandonaron sus estudios, que es la métrica de mayor relevancia práctica para esta aplicación, ya que minimiza la cantidad de estudiantes en riesgo que pasan desapercibidos.

5. EVALUACIÓN DEL MODELO

5.1 Métricas de evaluación

El modelo final (Random Forest optimizado) se evaluó utilizando las métricas estándar para problemas de clasificación binaria: precision, recall, F1-score y accuracy. Se dio especial énfasis al recall de la clase Dropout, ya que en este contexto es más costoso no detectar a un estudiante en riesgo (falso negativo) que generar una alerta sobre un estudiante que en realidad no abandonará (falso positivo).

Los resultados sobre el conjunto de prueba (1,328 estudiantes) fueron:

Clase	Precision	Recall	F1-score	Support
No Dropout	0.91	0.90	0.91	901
Dropout	0.80	0.82	0.81	427
Accuracy global	0.88			

5.2 Matriz de confusión

A partir de los valores de precision, recall y soporte reportados, la matriz de confusión aproximada sobre el conjunto de prueba es la siguiente:

	Predicho: No Dropout	Predicho: Dropout
Real: No Dropout	811	90
Real: Dropout	77	350

De los 427 estudiantes que efectivamente abandonaron, el modelo identificó correctamente a 350 (82 %), y clasificó erróneamente como "No Dropout" a 77 estudiantes en riesgo real. Por otro lado, de los 901 estudiantes que no abandonaron, 90 fueron señalados incorrectamente como en riesgo, lo cual es un costo aceptable si permite intervenir a tiempo con los verdaderos casos de riesgo.

5.3 Evaluación de resultados

El modelo muestra un desempeño sólido y equilibrado entre ambas clases, sin un sesgo marcado hacia la clase mayoritaria (No Dropout), lo cual confirma que el uso de `class_weight="balanced"` cumplió su propósito. El F1-score de 0.81 para la clase Dropout es consistente con lo

6. DESARROLLO DE LA APLICACIÓN

6.1 Tecnologías utilizadas

Para el desarrollo de la aplicación se utilizaron las siguientes tecnologías:

- **FastAPI**: framework para construir la API REST, elegido por su alto rendimiento, validación automática de datos mediante Pydantic, y generación automática de documentación interactiva (Swagger UI).
- **scikit-learn**: para el entrenamiento del modelo Random Forest.
- **SHAP**: para calcular la explicabilidad de las predicciones, tanto a nivel global (en el notebook de entrenamiento) como local, por cada predicción individual (en la API).
- **joblib**: para serializar y cargar el modelo entrenado y el orden de columnas esperado.
- **Streamlit**: framework utilizado para construir el frontend interactivo, que consume la API mediante peticiones HTTP.
- **Plotly**: para las visualizaciones del frontend (medidor de riesgo y gráfico de factores SHAP).
- **pytest**: para las pruebas automatizadas de la API.

6.2 Arquitectura de software

La aplicación sigue una arquitectura cliente-servidor simple, separando claramente el servicio de predicción (backend) de la interfaz de usuario (frontend). El backend expone una API REST documentada y versionada de forma independiente del frontend, lo que permite que ambos componentes se desplieguen, actualicen y escalen por separado. Esta separación también facilita que, en el futuro, otros clientes (por ejemplo, una aplicación móvil) puedan consumir la misma API sin modificaciones.

6.3 Integración del modelo de aprendizaje automático

El modelo entrenado se serializa con `joblib` en dos archivos: `modelo_desercion.pkl` (el modelo Random Forest) y `columnas.pkl` (el orden exacto de las 34 columnas que el modelo espera como entrada). Al iniciar la aplicación FastAPI, ambos archivos se cargan una única vez en memoria mediante un evento de *lifespan*, junto con un `shap.TreeExplainer`

inicializado sobre el modelo cargado. Esto evita recargar el modelo en cada petición, reduciendo significativamente el tiempo de respuesta de las predicciones.

Cuando llega una petición a `POST /predict`, los datos del estudiante se validan mediante un esquema Pydantic (`StudentFeatures`), se reordenan según el orden de columnas esperado, y se pasan al modelo para obtener la predicción y su probabilidad. Inmediatamente después, se calculan los valores SHAP correspondientes a esa predicción específica, y se seleccionan los tres factores con mayor valor absoluto de impacto para incluirlos en la respuesta.

6.4 Interfaz de la aplicación

El frontend, desarrollado en Streamlit, presenta un formulario dividido en seis secciones colapsables (datos personales y socioeconómicos, situación financiera y familiar, modalidad de matrícula, desempeño académico del primer semestre, desempeño académico del segundo semestre, y contexto macroeconómico), en lugar de mostrar los 34 campos de forma plana. Incluye además dos botones que cargan ejemplos predefinidos de estudiantes de alto y bajo riesgo, útiles tanto para pruebas rápidas como para la demostración del sistema.

Al enviar el formulario, el resultado se presenta mediante un medidor de riesgo (gauge) con código de colores según el nivel obtenido (bajo, medio o alto), y un gráfico de barras horizontal con los tres factores SHAP que más influyeron en esa predicción específica, indicando si cada uno empuja la predicción hacia mayor o menor riesgo de deserción.

6.5 Funcionamiento de la aplicación en inglés

Para cumplir con el requisito de que la aplicación funcione en inglés, el frontend incluye un selector de idioma (English / Español) ubicado en la parte superior de la interfaz. Al cambiar el idioma, se traducen tanto las etiquetas de cada campo del formulario como los resultados de la predicción (nivel de riesgo, probabilidad, factores SHAP), permitiendo demostrar el funcionamiento completo de la aplicación en inglés sin necesidad de mantener dos versiones separadas del sistema.

7. DESPLIEGUE EN PRODUCCIÓN

7.1 Entorno de despliegue

La aplicación se despliega en **Railway**, una plataforma de despliegue en la nube (PaaS) que permite ejecutar aplicaciones directamente desde un repositorio de GitHub, sin necesidad de gestionar servidores de forma manual. Se eligió Railway por su integración nativa con GitHub,

y su generación automática de un dominio público con HTTPS.

7.2 Configuración del entorno

El entorno de ejecución se define mediante tres archivos en la raíz del proyecto:

- `runtime.txt`: especifica la versión de Python utilizada (3.12).
- `requirements.txt`: lista las dependencias exactas del proyecto. Las versiones de `scikit-learn` y `xgboost` quedan fijas a las mismas usadas durante el entrenamiento del modelo, para evitar incompatibilidades al cargar el archivo `.pkl` en producción.
- `railway.json`: define el comando de arranque (`uvicorn app.main:app --host 0.0.0.0 --port $PORT`) y la política de reinicio en caso de fallo.

7.3 Contenerización

A diferencia de un despliegue tradicional basado en Docker, Railway utiliza **Nixpacks**, un sistema de build automático que detecta el lenguaje y las dependencias del proyecto (en este caso, a partir de `requirements.txt` y `runtime.txt`) y genera automáticamente un contenedor de ejecución, sin que el equipo deba escribir o mantener un `Dockerfile` manualmente. Este enfoque simplifica el despliegue para proyectos de este tamaño, delegando en la plataforma la construcción del entorno de ejecución.

7.4 Puesta en producción

El despliegue se realiza de forma continua: el repositorio de GitHub (`proyecto-desercion`) está conectado directamente al servicio en Railway, de modo que cada `git push` a la rama `main` dispara automáticamente un nuevo build y despliegue, sin intervención manual. La aplicación quedó disponible públicamente en: `https://web-production-ed3cf4.up.railway.app` y el frontend interactivo en: `https://proyecto-desercion-production.up.railway.app/`

7.5 Verificación del despliegue

Tras el despliegue, se verificó el correcto funcionamiento de la aplicación accediendo a los siguientes endpoints:

- `GET /`: responde con información básica de la API.

- `GET /health`: confirma que la API está activa y el modelo cargado correctamente en memoria.
- `GET /docs`: muestra la documentación interactiva (Swagger UI), donde se probó el endpoint `POST /predict` con datos reales de ejemplo, confirmando que el modelo genera predicciones correctas en el entorno de producción.

8. PLAN DE INTEGRACIÓN CONTINUA Y MANTENIMIENTO DEL MODELO

Este capítulo describe, de manera preliminar, los procesos de integración continua y de mantenimiento del modelo que se implementarán como parte de la entrega final del curso (Unidad II, 27 de setiembre de 2026). A la fecha de esta entrega, estos componentes aún no forman parte de la aplicación.

8.1 Integración continua (CI/CD)

Se planea implementar un flujo de integración continua mediante **GitHub Actions**, que ejecute automáticamente la suite de pruebas (`pytest`) ante cada *push* o *pull request* hacia la rama `main`. Solo si las pruebas pasan correctamente, se desencadenaría el despliegue automático a Railway, evitando que código con errores llegue a producción. Esto formalizaría el flujo de despliegue continuo que ya existe entre GitHub y Railway, añadiendo una validación automática de calidad antes de cada despliegue.

8.2 Mantenimiento del modelo

Para la Unidad II se plantea:

- **Versionamiento del modelo**: registrar cada versión entrenada junto con sus métricas de desempeño (F1, precision, recall) y la fecha de entrenamiento, mediante una base de datos ligera (PostgreSQL) o un registro de modelos simple.
- **Registro de predicciones**: almacenar cada predicción realizada en producción (entrada, resultado y timestamp) para permitir un monitoreo posterior del comportamiento del modelo con datos reales.
- **Reentrenamiento**: definir un proceso para reentrenar el modelo cuando se cuente con nuevos datos, comparando sus métricas contra la versión vigente antes de reemplazarla en producción.

- **Monitoreo:** supervisar métricas operativas de la API (tiempo de respuesta, tasa de errores) y señales de *data drift* que indiquen que el modelo podría necesitar reentrenamiento.

9. PRUEBAS DE FUNCIONAMIENTO

9.1 Diseño de pruebas

Las pruebas se implementaron con `pytest` y el `TestClient` de FastAPI, que permite probar la API sin necesidad de desplegarla, simulando peticiones HTTP directamente sobre la aplicación. Se diseñaron cinco pruebas que cubren los flujos principales: verificación de estado, predicción sobre casos conocidos de alto y bajo riesgo, validación de datos de entrada, y el endpoint raíz.

9.2 Prueba de funcionamiento 1: Health check

Verifica que la API esté activa y que el modelo se haya cargado correctamente en memoria.

```
def test_health_check():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"
    assert response.json()["model_loaded"] is True
```

Resultado: PASSED

9.3 Prueba de funcionamiento 2: Predicción sobre casos conocidos

Se probó el endpoint `/predict` con dos perfiles reales del dataset: un estudiante con indicadores claros de riesgo (deudor, matrícula atrasada, cero unidades aprobadas en el primer semestre) y un estudiante con buen desempeño académico y situación financiera regular.

- `test_predict_alto_riesgo`: verifica que el nivel de riesgo devuelto sea "Medio." "Alto" para el perfil adverso, y que la respuesta incluya los tres factores SHAP.
- `test_predict_bajo_riesgo`: verifica que el nivel de riesgo devuelto sea "Bajo." "Medio" para el perfil favorable.

Resultado: ambas PASSED

9.4 Pruebas adicionales

Se incluyeron dos pruebas complementarias:

- `test_predict_datos_invalidos`: envía una petición con datos incompletos y confirma que la API responde con código 422 (Unprocessable Entity), validando que Pydantic rechaza correctamente entradas mal formadas.
- `test_root_endpoint`: confirma que el endpoint raíz (GET /) responde con la información básica de la API.

Resultado: ambas PASSED

9.5 Resultados de las pruebas

La ejecución completa de la suite de pruebas mediante `pytest -v` arrojó el siguiente resultado:

```
tests/test_api.py::test_health_check PASSED
tests/test_api.py::test_predict_alto_riesgo PASSED
tests/test_api.py::test_predict_bajo_riesgo PASSED
tests/test_api.py::test_predict_datos_invalidos PASSED
tests/test_api.py::test_root_endpoint PASSED

===== 5 passed in 4.86s =====
```

Las cinco pruebas pasaron correctamente, confirmando el funcionamiento esperado de la API tanto en sus respuestas exitosas como en el manejo de entradas inválidas.

10. RESULTADOS

10.1 Resultados del modelo

El modelo final (Random Forest optimizado mediante `GridSearchCV`) alcanzó un F1-score de 0.81 y un recall de 0.82 para la clase Dropout sobre el conjunto de prueba, superando a Regresión Logística y XGBoost en la mayoría de las métricas evaluadas, como se detalló en el Capítulo V. El modelo entrenado se guardó como `modelo_desercion.pkl`, junto con el orden de columnas esperado en `columnas.pkl`, listos para ser consumidos por la API.

10.2 Resultados de la aplicación

Se desarrolló una API REST funcional con tres endpoints operativos (`/`, `/health`, `/predict`), documentada automáticamente mediante Swagger UI. Adicionalmente, se construyó un frontend interactivo en Streamlit que consume dicha API, con soporte bilingüe completo (inglés/español) tanto en las etiquetas del formulario como en las categorías de los campos desplegados, y con visualizaciones del resultado mediante un medidor de riesgo y un gráfico de factores SHAP.

10.3 Resultados del despliegue

La aplicación quedó desplegada y accesible públicamente en dos servicios independientes conectados entre sí:

API (FastAPI):

`https://web-production-ed3cf4.up.railway.app`

Frontend (Streamlit):

`https://proyecto-desercion-production.up.railway.app/`

Se verificó el funcionamiento correcto de los tres endpoints de la API en el entorno de producción, incluyendo una predicción de prueba ejecutada directamente desde la interfaz de Swagger UI (`https://web-production-ed3cf4.up.railway.app/docs`), confirmando que el modelo responde correctamente en el entorno desplegado y no solo en el entorno de desarrollo local. Asimismo, se verificó que el frontend en Streamlit consume correctamente la API desplegada y renderiza las predicciones con su respectivo nivel de riesgo.

10.4 Resultados de la integración continua

A la fecha de esta entrega, no se cuenta con un pipeline de integración continua implementado. El despliegue hacia Railway ya ocurre de forma automática ante cada *push* a la rama `main` del repositorio, pero sin una etapa previa de validación automática mediante pruebas. La implementación de dicha validación mediante GitHub Actions se plantea como parte del trabajo descrito en el Capítulo VII (Plan de Integración Continua y Mantenimiento del Modelo), a desarrollarse para la entrega final de la Unidad II.

11. CONCLUSIONES

- Se logró desarrollar un modelo de clasificación capaz de predecir el riesgo de deserción estudiantil con un F1-score de 0.81 y un recall de 0.82 para la clase Dropout, lo que indica

que el modelo identifica correctamente a la mayoría de los estudiantes que efectivamente abandonan sus estudios.

- La elección de Random Forest sobre modelos con métricas ligeramente superiores en validación cruzada (como Regresión Logística) resultó adecuada, ya que permitió incorporar explicabilidad mediante SHAP sin sacrificar capacidad predictiva, cumpliendo así un requisito central del proyecto: que cada predicción pueda ser justificada, no solo entregada como una salida sin contexto.
- El análisis SHAP confirmó que los factores más influyentes en el riesgo de deserción están relacionados con el desempeño académico temprano (unidades aprobadas y notas en los dos primeros semestres) y con la situación financiera del estudiante (matrícula al día), lo cual es consistente con estudios previos sobre deserción universitaria.
- Se logró exponer el modelo mediante una API REST funcional, desplegada en un entorno de producción real (Railway) y accesible públicamente, cumpliendo con el objetivo de llevar el modelo más allá del entorno de experimentación en notebook.
- El desarrollo de un frontend interactivo y bilingüe permitió que la aplicación no dependiera únicamente de la documentación técnica de la API (Swagger UI), sino que pudiera ser utilizada por personal sin conocimientos de programación, acercando el proyecto a un caso de uso realista.
- Las pruebas automatizadas implementadas permitieron verificar de forma consistente el comportamiento esperado de la API, tanto en predicciones válidas como en el manejo de datos de entrada incorrectos, sentando una base sobre la cual se puede construir el pipeline de integración continua planificado para la Unidad II.
- Esta entrega deja pendientes, de forma intencional y declarada, los procesos de integración continua y mantenimiento del modelo, los cuales corresponden al alcance de la entrega final del curso.

12. RECOMENDACIONES

- Se recomienda migrar el despliegue de Nixpacks a un `Dockerfile` explícito para la entrega final, con el fin de ganar mayor control y portabilidad sobre el entorno de ejecución, permitiendo probar el mismo contenedor de forma idéntica en desarrollo local y en producción.
- Se recomienda implementar el pipeline de integración continua (GitHub Actions) antes de agregar nuevas funcionalidades a la aplicación, de modo que cualquier cambio futuro quede automáticamente respaldado por la suite de pruebas existente.

- Dado que no se identificaron conjuntos de datos abiertos sobre deserción universitaria peruana a nivel de estudiante individual, se recomienda, como línea de trabajo futura fuera del alcance de este curso, gestionar el acceso a datos anonimizados de alguna universidad peruana, con el fin de validar si el modelo entrenado con el dataset portugués generaliza adecuadamente al contexto local.
- Se recomienda revisar y fijar con mayor precisión la versión de `scikit-learn` utilizada durante el entrenamiento del modelo, ya que se detectó una advertencia de incompatibilidad de versión al cargar el modelo serializado en el entorno de pruebas.
- Para la Unidad II, se recomienda incorporar un mecanismo de registro de predicciones en producción (por ejemplo, mediante PostgreSQL), que permita evaluar en el tiempo si el desempeño del modelo se mantiene estable o si comienza a degradarse (*data drift*), antes de que dicho deterioro sea evidente para los usuarios finales de la aplicación.
- Se recomienda extender las pruebas automatizadas actuales con casos adicionales que cubran los límites de las validaciones de Pydantic (por ejemplo, edades fuera de rango o valores negativos en unidades curriculares), fortaleciendo la cobertura de pruebas de cara a la implementación del pipeline de integración continua.

ANEXOS

El código fuente completo del proyecto está disponible públicamente en:

<https://github.com/darig7w7/proyecto-desercion>

ANEXO 01. Diccionario de variables del dataset

La Tabla 12.1 describe las 34 variables predictoras del dataset *Predict Students' Dropout and Academic Success* (Realinho et al., 2021), utilizadas para el entrenamiento del modelo.

Variable	Descripción
Marital status	Estado civil del estudiante (código 1-6)
Application mode	Modalidad de postulación a la carrera (código)
Application order	Orden de preferencia de la postulación (0-9)
Course	Código de la carrera profesional
Daytime/evening attendance	Turno de asistencia: 1 = Diurno, 0 = Vespertino
Previous qualification	Código de la calificación académica previa
Nationality	Código de nacionalidad
Mother's qualification	Código del nivel educativo de la madre
Father's qualification	Código del nivel educativo del padre
Mother's occupation	Código de la ocupación de la madre
Father's occupation	Código de la ocupación del padre
Displaced	Estudiante desplazado de su lugar de origen: 1 = Sí, 0 = No
Educational special needs	Necesidades educativas especiales: 1 = Sí, 0 = No
Debtor	Estudiante con deudas pendientes: 1 = Sí, 0 = No
Tuition fees up to date	Pago de matrícula al día: 1 = Sí, 0 = No
Gender	Género: 1 = Hombre, 0 = Mujer
Scholarship holder	Becario: 1 = Sí, 0 = No
Age at enrollment	Edad al momento de matricularse
International	Estudiante internacional: 1 = Sí, 0 = No
Curricular units 1st sem (credited)	Unidades curriculares convalidadas, 1er semestre
Curricular units 1st sem (enrolled)	Unidades curriculares matriculadas, 1er semestre
Curricular units 1st sem (evaluations)	Unidades curriculares evaluadas, 1er semestre
Curricular units 1st sem (approved)	Unidades curriculares aprobadas, 1er semestre

Variable	Descripción
Curricular units 1st sem (grade)	Nota promedio, 1er semestre
Curricular units 1st sem (without evaluations)	Unidades curriculares sin evaluación, 1er semestre
Curricular units 2nd sem (credited)	Unidades curriculares convalidadas, 2do semestre
Curricular units 2nd sem (enrolled)	Unidades curriculares matriculadas, 2do semestre
Curricular units 2nd sem (evaluations)	Unidades curriculares evaluadas, 2do semestre
Curricular units 2nd sem (approved)	Unidades curriculares aprobadas, 2do semestre
Curricular units 2nd sem (grade)	Nota promedio, 2do semestre
Curricular units 2nd sem (without evaluations)	Unidades curriculares sin evaluación, 2do semestre
Unemployment rate	Tasa de desempleo nacional (%) al momento de matrícula
Inflation rate	Tasa de inflación nacional (%) al momento de matrícula
GDP	Producto Bruto Interno (variación %) al momento de matrícula

Tabla 12.1: Diccionario de variables del dataset

ANEXO 02. Código fuente de la API

app/main.py

```
1 """
2 import os
3 from contextlib import asynccontextmanager
4
5 import joblib
6 import numpy as np
7 import pandas as pd
8 import shap
9 from fastapi import FastAPI, HTTPException
10 from fastapi.middleware.cors import CORSMiddleware
11 from app.schemas import StudentFeatures, PredictionResponse,
    HealthResponse
12 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
13 MODEL_PATH = os.path.join(BASE_DIR, "models", "modelo_desercion.pkl")
14 COLUMNS_PATH = os.path.join(BASE_DIR, "models", "columnas.pkl")
15 @asynccontextmanager
16 async def lifespan(app: FastAPI):
17     _cargar_modelo()
18     yield
19 app = FastAPI(
20     title="API de Prediccion de Desercion Estudiantil",
21     description=(
22         "Predice el riesgo de desercion de un estudiante
23         universitario a partir de "
24         "datos demograficos, socioeconomicos y de desempeno
25         academico. "
26         "Proyecto del curso Aprendizaje de Maquina I - UNA Puno."
27     ),
28     version="1.0.0",
29     lifespan=lifespan,
30 )
31 app.add_middleware(
32     CORSMiddleware,
33     allow_origins=["*"],
34     allow_methods=["*"],
35     allow_headers=["*"],
36 )
37 _modelo = None
38 _columnas = None
39 _explainer = None
40 def _cargar_modelo():
41     global _modelo, _columnas, _explainer
42     if _modelo is None:
```

```

41     _modelo = joblib.load(MODEL_PATH)
42     _columnas = joblib.load(COLUMNS_PATH)
43     _explainer = shap.TreeExplainer(_modelo)
44     return _modelo, _columnas, _explainer
45 @app.get("/", tags=["General"])
46 def root():
47     return {
48         "mensaje": "API de Prediccion de Desercion Estudiantil",
49         "docs": "/docs",
50         "health": "/health",
51     }
52 @app.get("/health", response_model=HealthResponse, tags=["General"])
53 def health():
54     try:
55         modelo, _, _ = _cargar_modelo()
56         return HealthResponse(status="ok", model_loaded=modelo is
57                               not None)
58     except Exception:
59         return HealthResponse(status="error", model_loaded=False)
60 def _clasificar_riesgo(prob: float) -> str:
61     if prob < 0.33:
62         return "Bajo"
63     elif prob < 0.66:
64         return "Medio"
65     return "Alto"
66 @app.post("/predict", response_model=PredictionResponse, tags=["
67     Prediccion"])
68 def predict(student: StudentFeatures):
69     try:
70         modelo, columnas, explainer = _cargar_modelo()
71         datos = student.model_dump(by_alias=True)
72         fila = pd.DataFrame([datos])[columnas]
73         pred = modelo.predict(fila)[0]
74         proba = modelo.predict_proba(fila)[0][1]
75         shap_values = explainer.shap_values(fila)
76         if isinstance(shap_values, list):
77             sv = shap_values[1][0]
78         elif np.ndim(shap_values) == 3:
79             sv = shap_values[0, :, 1]
80         else:
81             sv = shap_values[0]
82         contribuciones = sorted(
83             zip(columnas, sv), key=lambda x: abs(x[1]), reverse=True
84        )[:3]
85         top_factors = [
86             {"feature": nombre, "impacto": round(float(valor), 4)}
87             for nombre, valor in contribuciones
88         ]
89         return PredictionResponse(
90             prediction="Dropout" if pred == 1 else "No Dropout",
91             dropout_probability=round(float(proba), 4),

```

```

90         risk_level=_clasificar_riesgo(proba),
91         top_factors=top_factors,
92     )
93 except Exception as e:
94     raise HTTPException(status_code=500, detail=f"Error al
        predecir: {str(e)}")

```

app/schemas.py

```

1 from pydantic import BaseModel, Field, ConfigDict
2 class StudentFeatures(BaseModel):
3     model_config = ConfigDict(populate_by_name=True)
4     marital_status: int = Field(..., alias="Marital status", ge=1,
        le=6)
5     application_mode: int = Field(..., alias="Application mode")
6     application_order: int = Field(..., alias="Application order",
        ge=0, le=9)
7     course: int = Field(..., alias="Course")
8     daytime_evening_attendance: int = Field(..., alias="Daytime/
        evening attendance")
9     previous_qualification: int = Field(..., alias="Previous
        qualification")
10    nationality: int = Field(..., alias="Nacionality")
11    mothers_qualification: int = Field(..., alias="Mother's
        qualification")
12    fathers_qualification: int = Field(..., alias="Father's
        qualification")
13    mothers_occupation: int = Field(..., alias="Mother's occupation"
        )
14    fathers_occupation: int = Field(..., alias="Father's occupation"
        )
15    displaced: int = Field(..., alias="Displaced")
16    educational_special_needs: int = Field(..., alias="Educational
        special needs")
17    debtor: int = Field(..., alias="Debtor")
18    tuition_fees_up_to_date: int = Field(..., alias="Tuition fees up
        to date")
19    gender: int = Field(..., alias="Gender")
20    scholarship_holder: int = Field(..., alias="Scholarship holder")
21    age_at_enrollment: int = Field(..., alias="Age at enrollment",
        ge=15, le=90)
22    international: int = Field(..., alias="International")
23    curricular_units_1st_sem_credited: int = Field(..., alias="
        Curricular units 1st sem (credited)")
24    curricular_units_1st_sem_enrolled: int = Field(..., alias="
        Curricular units 1st sem (enrolled)")
25    curricular_units_1st_sem_evaluations: int = Field(..., alias="
        Curricular units 1st sem (evaluations)")

```

```

26     curricular_units_1st_sem_approved: int = Field(..., alias="
        Curricular units 1st sem (approved)")
27     curricular_units_1st_sem_grade: float = Field(..., alias="
        Curricular units 1st sem (grade)")
28     curricular_units_1st_sem_without_evaluations: int = Field(
29         ..., alias="Curricular units 1st sem (without evaluations)")
30     curricular_units_2nd_sem_credited: int = Field(..., alias="
        Curricular units 2nd sem (credited)")
31     curricular_units_2nd_sem_enrolled: int = Field(..., alias="
        Curricular units 2nd sem (enrolled)")
32     curricular_units_2nd_sem_evaluations: int = Field(..., alias="
        Curricular units 2nd sem (evaluations)")
33     curricular_units_2nd_sem_approved: int = Field(..., alias="
        Curricular units 2nd sem (approved)")
34     curricular_units_2nd_sem_grade: float = Field(..., alias="
        Curricular units 2nd sem (grade)")
35     curricular_units_2nd_sem_without_evaluations: int = Field(
36         ..., alias="Curricular units 2nd sem (without evaluations)")
37     unemployment_rate: float = Field(..., alias="Unemployment rate")
38     inflation_rate: float = Field(..., alias="Inflation rate")
39     gdp: float = Field(..., alias="GDP")
40 class PredictionResponse(BaseModel):
41     prediction: str = Field(..., description='"Dropout" o "No
        Dropout"')
42     dropout_probability: float = Field(..., description="
        Probabilidad de abandono (0 a 1)")
43     risk_level: str = Field(..., description='"Bajo", "Medio" o "
        Alto"')
44     top_factors: list[dict] = Field(default_factory=list)
45 class HealthResponse(BaseModel):
46     status: str
47     model_loaded: bool

```

entrenamiento.ipynb

```

1  ## CARGA Y PREPARACION DE DATOS
2
3  df = pd.read_csv("dataset.csv")
4  print(f"Shape: {df.shape}")
5  df.head()
6  # Variable binaria: 1 = Dropout, 0 = No Dropout (Enrolled + Graduate
   )
7  df['dropout'] = (df['Target'] == 'Dropout').astype(int)
8  df = df.drop(columns=['Target'])
9  print(df['dropout'].value_counts())
10 print(f"\nProporcion de abandono: {df['dropout'].mean():.2%}")
11 X = df.drop(columns=['dropout'])
12 y = df['dropout']
13 X_train, X_test, y_train, y_test = train_test_split(

```

```

14 X, y, test_size=0.3, stratify=y, random_state=SEED
15 )
16 scaler = StandardScaler()
17 X_train_s = scaler.fit_transform(X_train)
18 X_test_s = scaler.transform(X_test)
19 print(f"Train: {X_train.shape}, Test: {X_test.shape}")

```

```

1 ## COMPARACION DE MODELOS
2
3 def evaluar(nombre, modelo, X_tr, y_tr, X_te, y_te, resultados):
4     modelo.fit(X_tr, y_tr)
5     y_pred = modelo.predict(X_te)
6     cv = cross_val_score(modelo, X_tr, y_tr, cv=5, scoring="f1").
7         mean()
8     resultados.append({
9         "Modelo": nombre,
10        "Accuracy": accuracy_score(y_te, y_pred),
11        "Precision": precision_score(y_te, y_pred),
12        "Recall": recall_score(y_te, y_pred),
13        "F1": f1_score(y_te, y_pred),
14        "CV F1 (5-fold)": cv,
15    })
16    return modelo
17 resultados = []
18 logreg = LogisticRegression(max_iter=5000, random_state=SEED,
19                             class_weight="balanced")
20 evaluar("Regresion Logistica", logreg, X_train_s, y_train, X_test_s,
21        y_test, resultados)
22 rf = RandomForestClassifier(n_estimators=300, max_depth=None,
23                             random_state=SEED,
24                             class_weight="balanced", n_jobs=-1)
25 evaluar("Random Forest", rf, X_train, y_train, X_test, y_test,
26        resultados)
27 # scale_pos_weight ajusta XGBoost al desbalance de clases
28 spw = (y_train == 0).sum() / (y_train == 1).sum()
29 xgb = XGBClassifier(n_estimators=300, max_depth=4, learning_rate
30                    =0.1,
31                    eval_metric="logloss", random_state=SEED,
32                    scale_pos_weight=spw, n_jobs=-1)
33 evaluar("XGBoost", xgb, X_train, y_train, X_test, y_test, resultados
34        )
35 df_resultados = pd.DataFrame(resultados).sort_values("F1", ascending
36              =False).reset_index(drop=True)
37 df_resultados

```

```

1 ## OPTIMIZACION DE PARAMETROS
2
3 # Grid Search sobre Random Forest (ajustar si el mejor modelo fue
4   otro)
5 param_grid = {

```

```

5     "n_estimators": [200, 300, 400],
6     "max_depth": [8, 12, None],
7     "min_samples_leaf": [1, 3, 5],
8 }
9 grid = GridSearchCV(
10     RandomForestClassifier(random_state=SEED, class_weight="balanced
11     ", n_jobs=-1),
12     param_grid=param_grid,
13     scoring="f1",
14     cv=5,
15     n_jobs=-1
16 )
17 grid.fit(X_train, y_train)
18 print("Mejores hiperparametros:", grid.best_params_)
19 print(f"Mejor F1 en CV: {grid.best_score_:.4f}")
20 modelo_final = grid.best_estimator_
21 y_pred_final = modelo_final.predict(X_test)
22
23 print("\nReporte de clasificacion (conjunto de prueba):")
24 print(classification_report(y_test, y_pred_final, target_names=["No
25     Dropout", "Dropout"]))

```

```

1 ## EXPLICABILIDAD DEL MODELO "SHAP"
2
3 explainer = shap.TreeExplainer(modelo_final)
4 shap_values = explainer.shap_values(X_test)
5 # Para clasificacion binaria, shap_values puede venir como lista [
6     clase0, clase1]
7 # o array 3D segun version
8 if isinstance(shap_values, list):
9     sv_dropout = shap_values[1]
10 else:
11     sv_dropout = shap_values[:, :, 1] if shap_values.ndim == 3 else
12     shap_values
13 shap.summary_plot(sv_dropout, X_test, show=False)
14 plt.tight_layout()
15 plt.show()

```

```

1 ## GUARDADO DEL MODELO FINAL
2
3 import os
4 os.makedirs("../models", exist_ok=True)
5 joblib.dump(modelo_final, "../models/modelo_desercion.pkl")
6 joblib.dump(list(X.columns), "../models/columnas.pkl")
7 print("Modelo guardado en ../models/modelo_desercion.pkl")
8 print(f"Numero de caracteristicas esperadas por el modelo: {len(X.
9     columns)}")

```

ANEXO 03. Configuración de despliegue

A continuación se presentan los archivos de configuración utilizados para desplegar la aplicación en la plataforma Railway, conectada al repositorio de GitHub del proyecto para redesplicue automático en cada *push* a la rama `main`.

Procfile

```
1 web: unicorn app.main:app --host 0.0.0.0 --port $PORT
```

railway.json

```
1 {  
2   "$schema": "https://railway.app/railway.schema.json",  
3   "build": {  
4     "builder": "NIXPACKS"  
5   },  
6   "deploy": {  
7     "startCommand": "unicorn app.main:app --host 0.0.0.0 --port  
8       $PORT",  
9     "restartPolicyType": "ON_FAILURE",  
10    "restartPolicyMaxRetries": 3  
11  }
```

runtime.txt

```
1 python-3.12
```

ANEXO 04. Pruebas

Interfaz de usuario (Streamlit)

Figura 12.1: Formulario de predicción con ejemplo para bajo riesgo con su resultado y factores influyentes

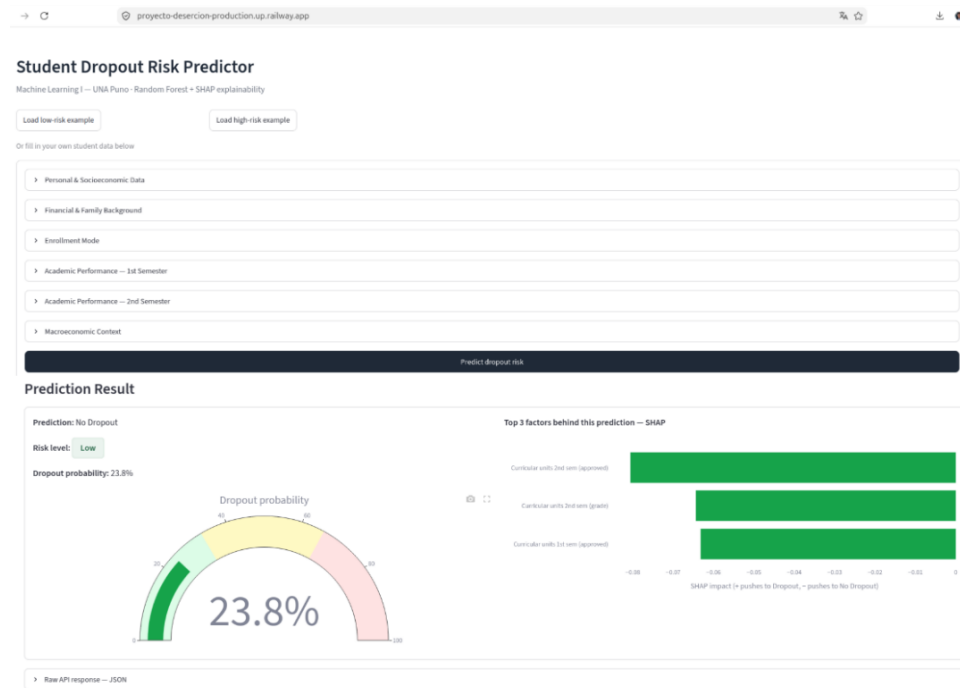
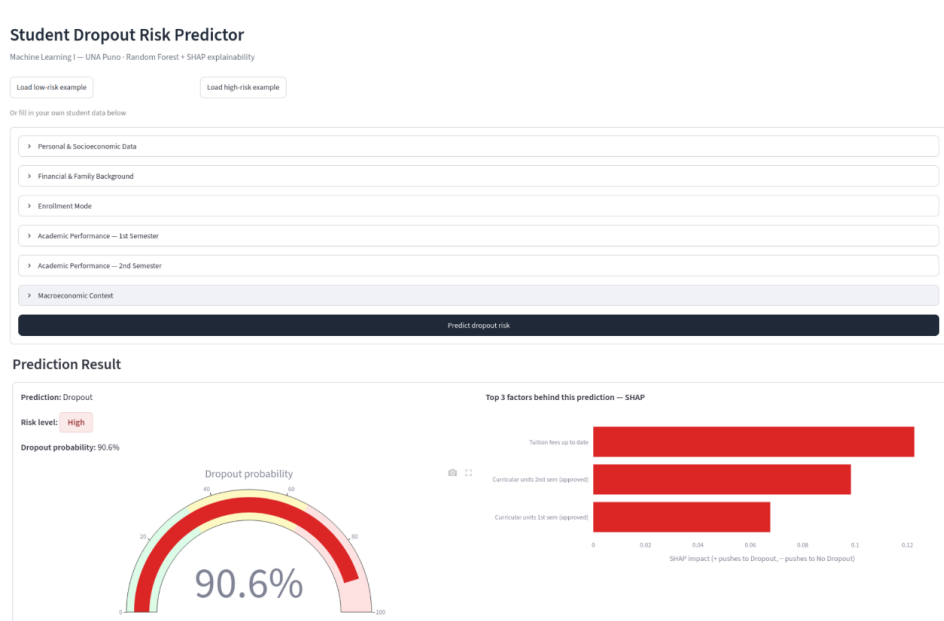


Figura 12.2: Formulario de predicción con ejemplo para alto riesgo con su resultado y factores influyentes

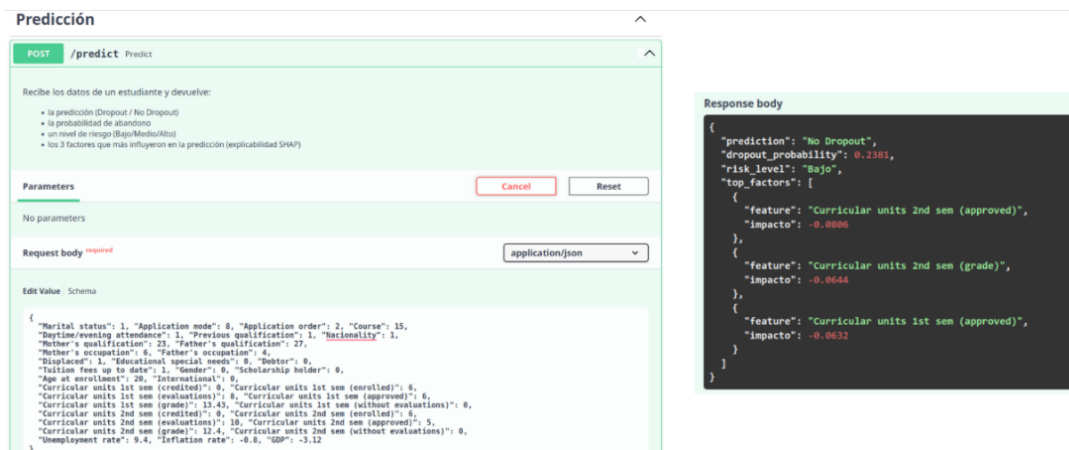


Vista general de endpoints

Figura 12.3: Endpoints listados (GET /, GET /health, POST /predict)



Figura 12.4: Panel de despliegue en Railway mostrando el servicio activo y el dominio público asignado



Despliegue

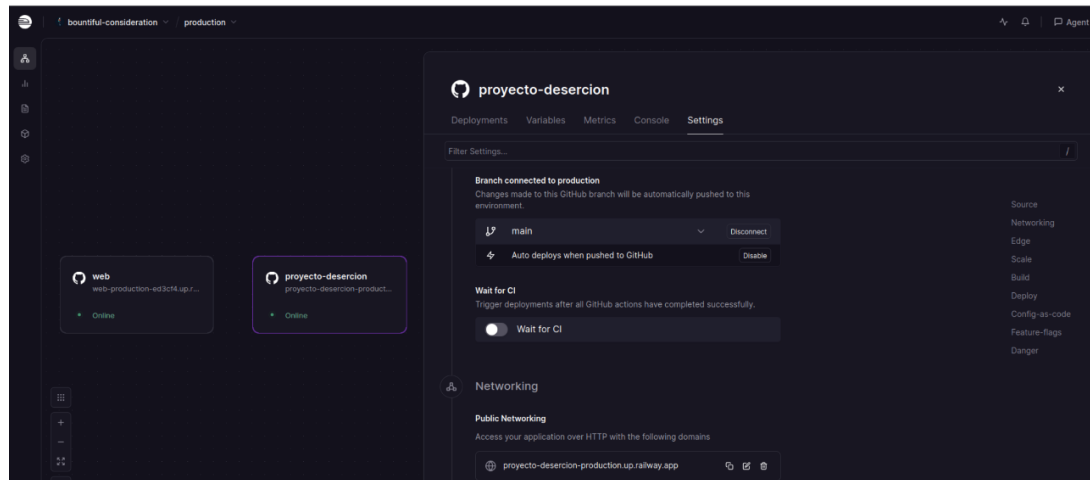


Figura 12.5: Panel de despliegue en Railway mostrando el servicio activo y el dominio público asignado